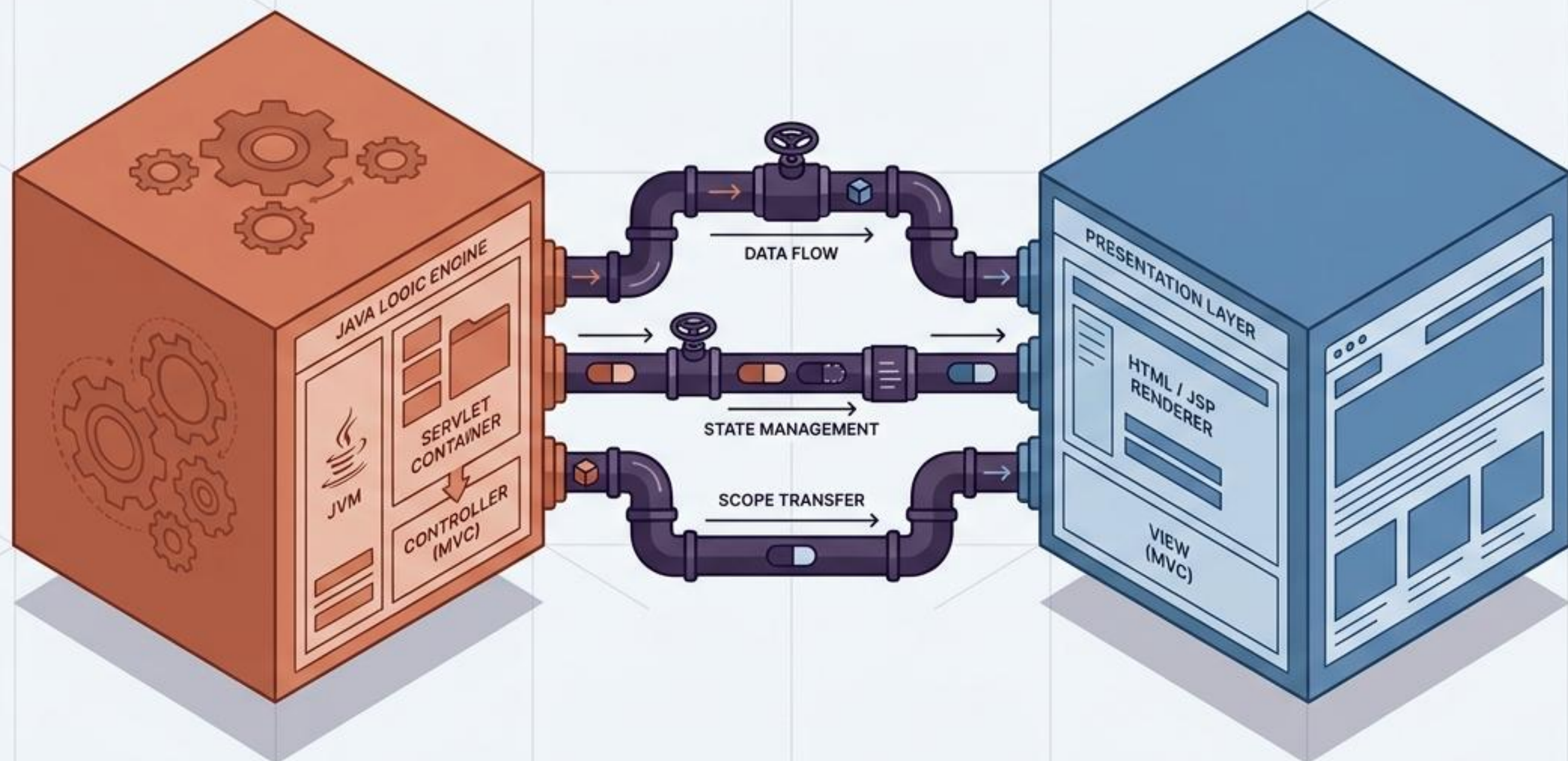


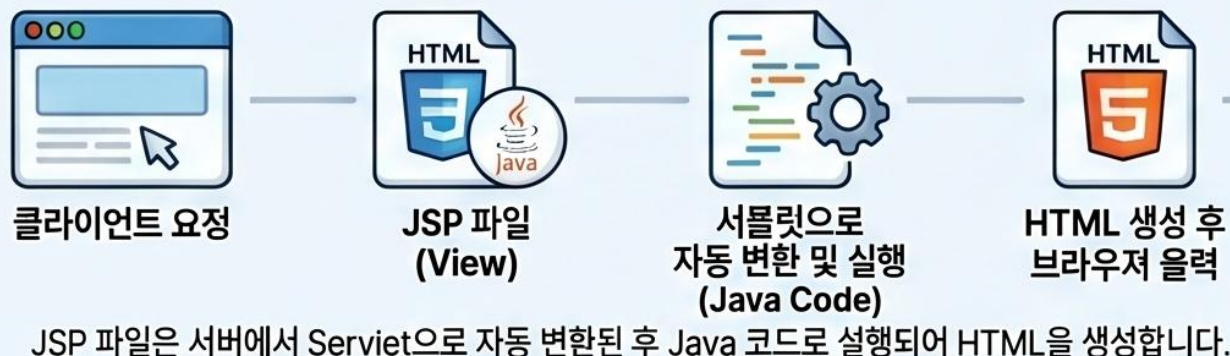
자바 웹 개발의 설계도

Servlet과 JSP, 그리고 MVC 아키텍처의 완벽한 이해



비전공자를 위한 JSP 웹 프로그래밍 핵심 가이드

JSP가 화면에 나타나는 과정



웹 서버와 WAS(Tomcat)의 협업



데이터 저장 방식: 쿠키 vs 세션

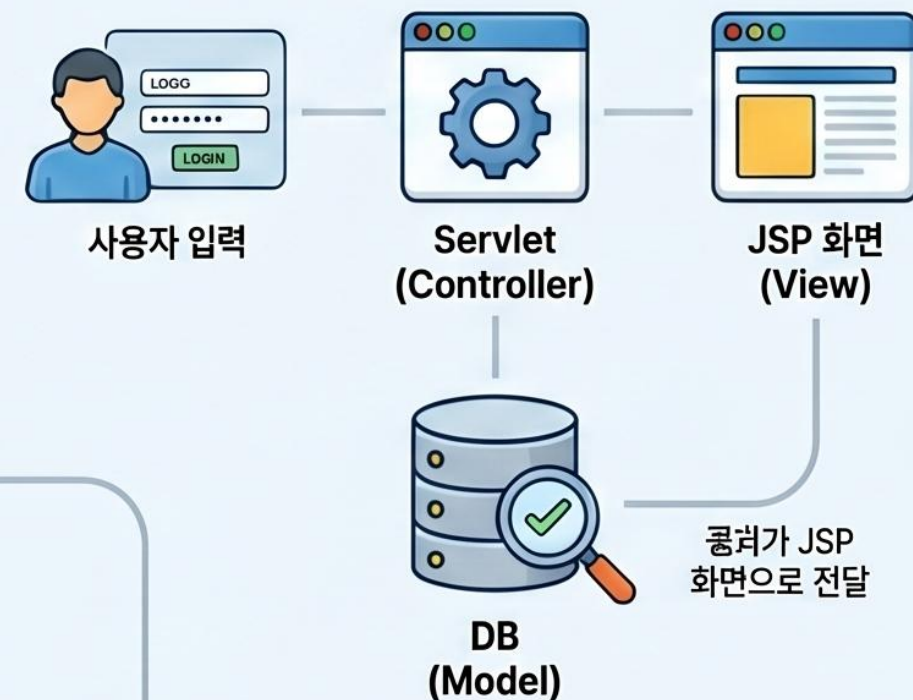


JSP와 Servlet의 근본적인 차이점 비교

구분	JSP	서블릿(Servlet)
주요 역할	화면 구현 (View)	비즈니스 로직 처리 (Controller)
형태	HTML 기만 + Java 코드	순수 Java 코드 기만
실행 방식	Servlet으로 변환 후 실행	즉시 실행 가능

웹 시스템의 완성, MVC 패턴

로그인으로 보는 시스템 흐름



View (화면)



역할 분담의 핵심, MVC

Model(데이터), View(화면), Controller(차리)로 역할을 나누어 유지보수를 쉽게 합니다.



Model (데이터)

효율적인 개발을 위한 모듈화



include

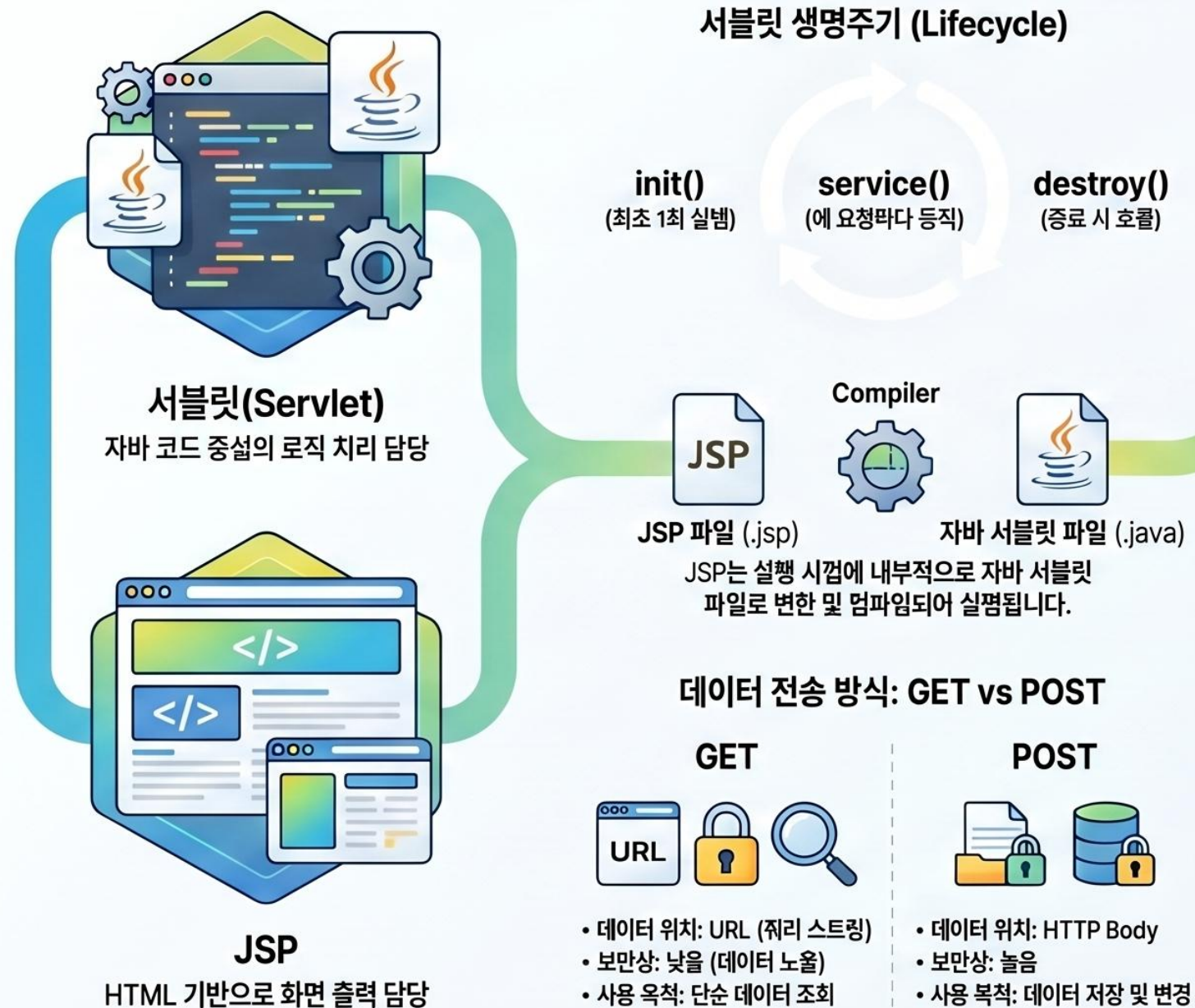


Java Bean

include와 Java Bean을 사용하여 코드 중복을 줄이고 데이터를 체계적으로 관리합니다.

Servlet & JSP: 웹 개발 핵심 메커니즘 총정리

서블릿과 JSP의 기초 및 생명주기



MVC 패턴과 데이터 흐름



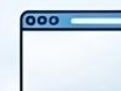
흐름 제어: Forward vs Redirect

Forward



서버 내부 이동
(URL 변하지 않음)

Redirect



URL을 변경하여 재요청

데이터 저장 영역 (Scope)



상태 유지: Session vs Cookie



Session (세션)

- 저장 위치: 서버 (Server)
- 보안 수준: 높음
- 유지 범위: 브라우저 종료 시까지



Cookie (쿠키)

- 저장 위치: 브라우저 (Client)
- 보안 수준: 낮음
- 유지 범위: 설정한 유효 시간 동안

웹을 구동하는 두 가지 엔진



Servlet

- 자바 기반 서버 프로그램 (Java 코드 중심)
- 요청 처리 및 비즈니스 로직 담당
- HTML 출력에는 비효율적 (유지보수 어려움)
- 역할: Controller ('두뇌')



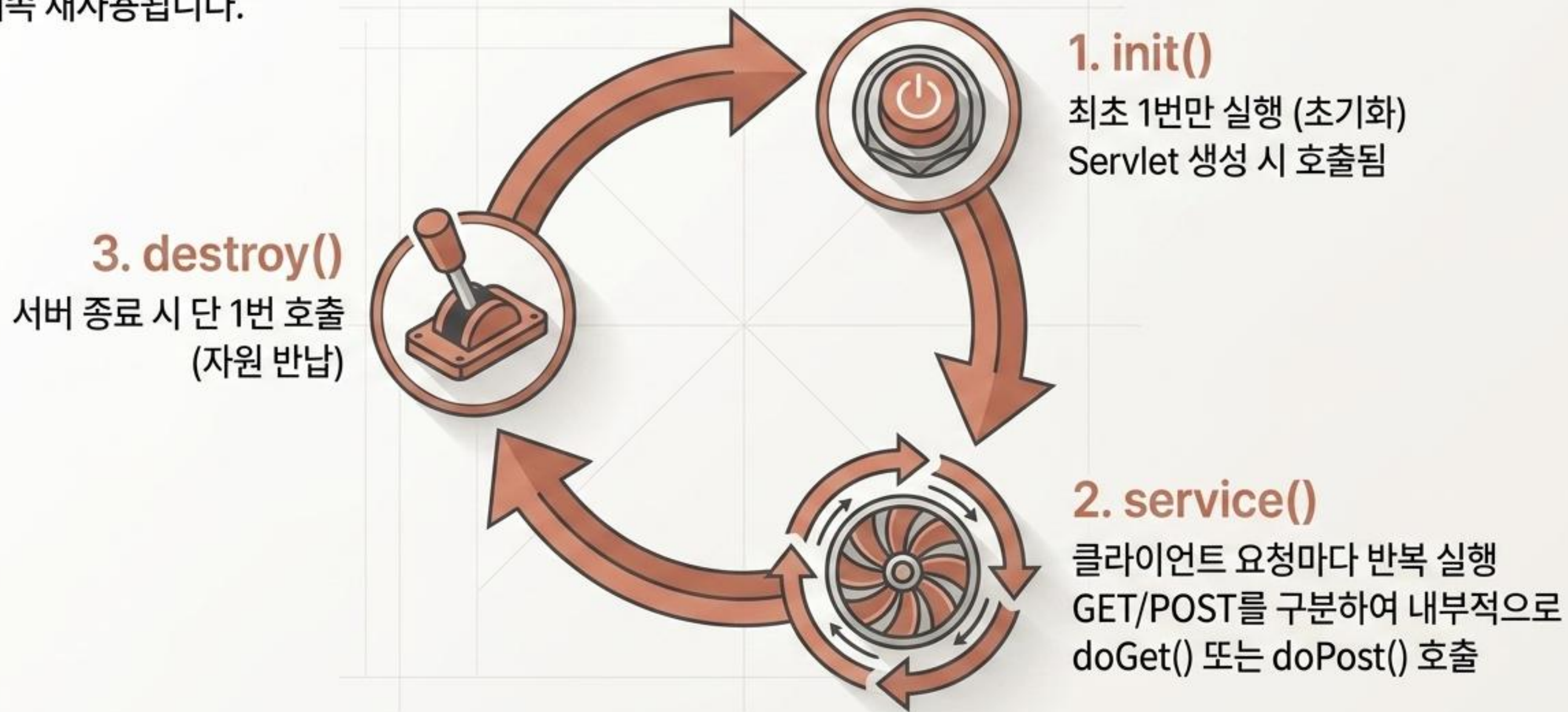
JSP (Java Server Pages)

- HTML 내부에 Java 코드 삽입 (화면 중심)
- 클라이언트가 볼 화면 구성 담당
- 복잡한 로직 처리에는 비효율적
- 역할: View ('얼굴')

결론: Servlet은 로직에 강하고, JSP는 화면에 강하다.

Servlet 생명주기 (Lifecycle)

Servlet 객체는 최초 한 번만 생성되며,
이후 계속 재사용됩니다.



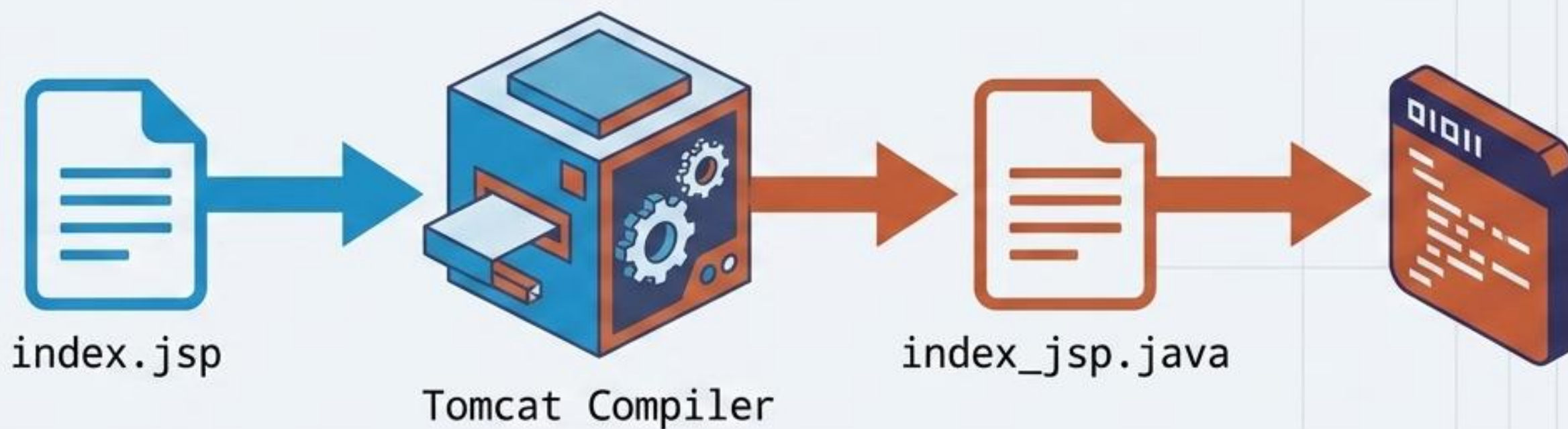
데이터 전달 방식: GET vs POST



구분	GET	POST
데이터 위치	URL 뒤에 노출 (Query String)	HTTP Body 내부에 숨김
보안	낮음 (노출됨)	높음 (안전함)
길이 제한	있음	거의 없음
사용 목적	데이터 '조회' (Read)	데이터 '저장/수정' (Write)

```
protected void service(HttpServletRequest req, HttpServletResponse resp)
// 내부에서 요청 방식에 따라 doGet() 또는 doPost()로 분기 처리
```

JSP의 숨겨진 진실: 'JSP는 결국 Servlet이다'



1. 최초 요청 시 브라우저가 `index.jsp` 호출
2. Tomcat이 JSP 파일을 Servlet(`.java`)으로 변환
3. 변환된 Java 코드가 컴파일되어 실행

`index.jsp` ➡ Tomcat Compiler ➡ `index_jsp.java` ➡ 컴파일 및 실행
이 변환 과정 때문에 JSP의 '최초' 요청은 미세하게 느리게 동작합니다.

JSP 기본 문법과 진화: 코드를 최소화하라

Java 로직은 Servlet으로 넘기고, JSP는 '출력'만 담당합니다.

과거 방식: Scriptlet



```
<% // 비즈니스 로직과 화면이 섞여 유지보수 최악  
    int a = 10;  
    Connection conn = ...  
%>
```

출력: <%= a %>

권장 방식: EL & JSTL



EL (Expression Language):

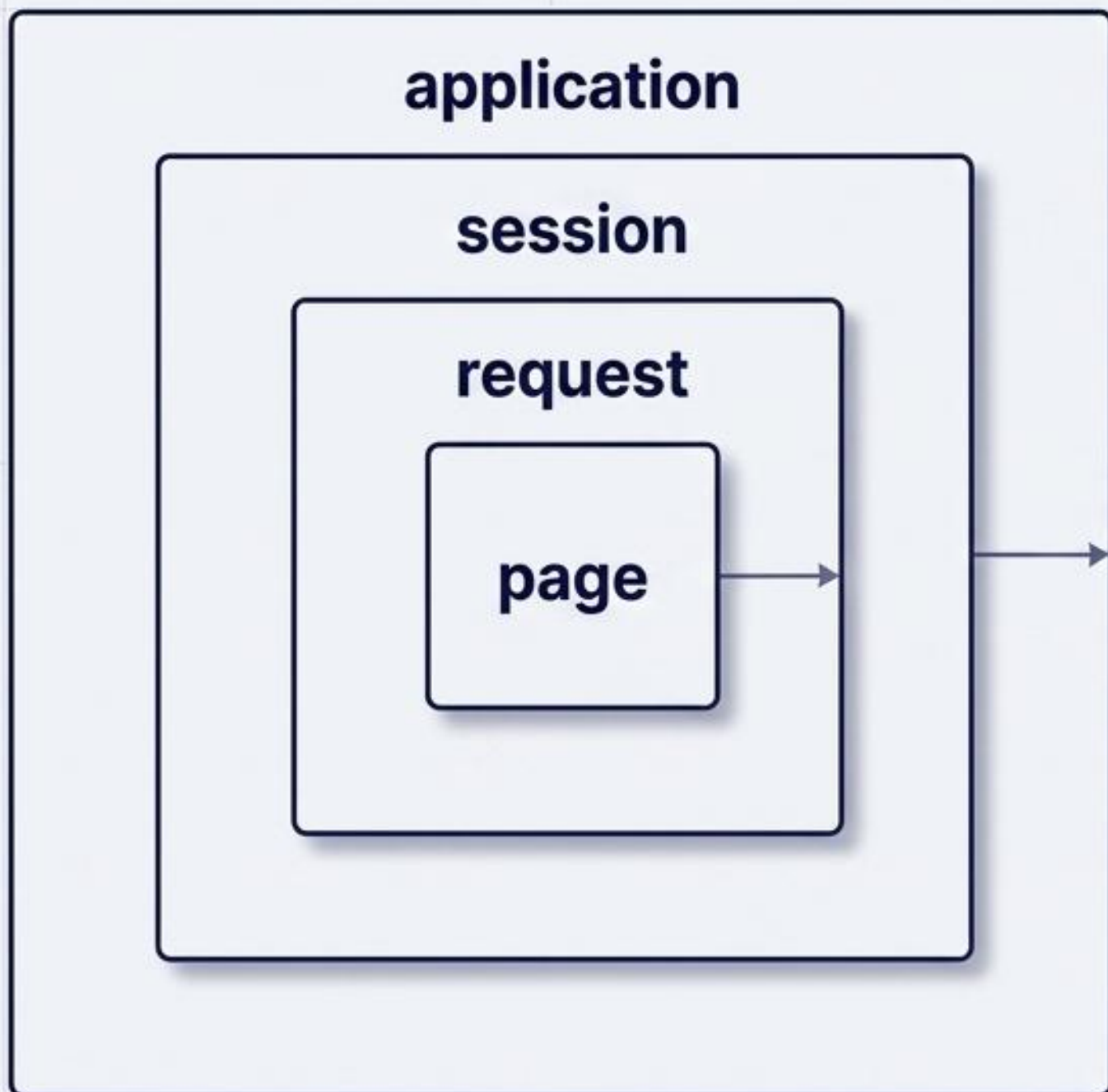
`${member.id}`

`${sessionScope.login}`

JSTL (JSP Standard Tag Library):

```
<c:if test='${num > 10}'> 크다 </c:if>  
<c:if test='${num > 10}'> 크다 </c:if>
```

데이터의 생존 범위: Scope (영역)



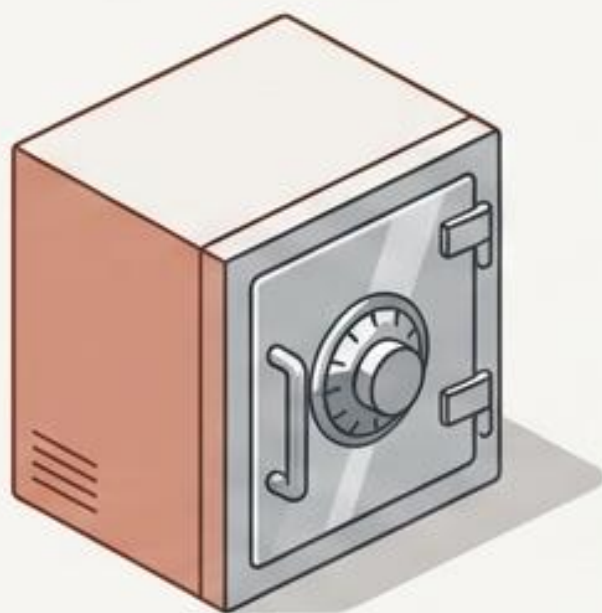
- **page** : 현재 머물고 있는 단일 JSP 페이지 내에서만 유효
- **request** : 하나의 클라이언트 요청이 처리되는 동안 유지 (Forward 시 유지됨)
- **session** : 브라우저가 열려있는 동안(사용자 로그인 상태 등) 지속 유지
- **application** : Tomcat 서버가 켜져 있는 동안 전체에서 공유되는 전역 공간

데이터 저장 방식:

```
request.setAttribute('name', 'kim');
```

상태 유지 기술: Session vs Cookie

HTTP는 기본적으로 상태를 기억하지 못합니다(Stateless). 따라서 사용자를 기억할 도구가 필요합니다.



Session: 서버 금고

- 저장 위치: 서버 메모리
- 보안성: 매우 높음 (중요 데이터 저장)
- 사용 예시: 사용자 로그인 정보

```
session.setAttribute('login', 'kim');
```

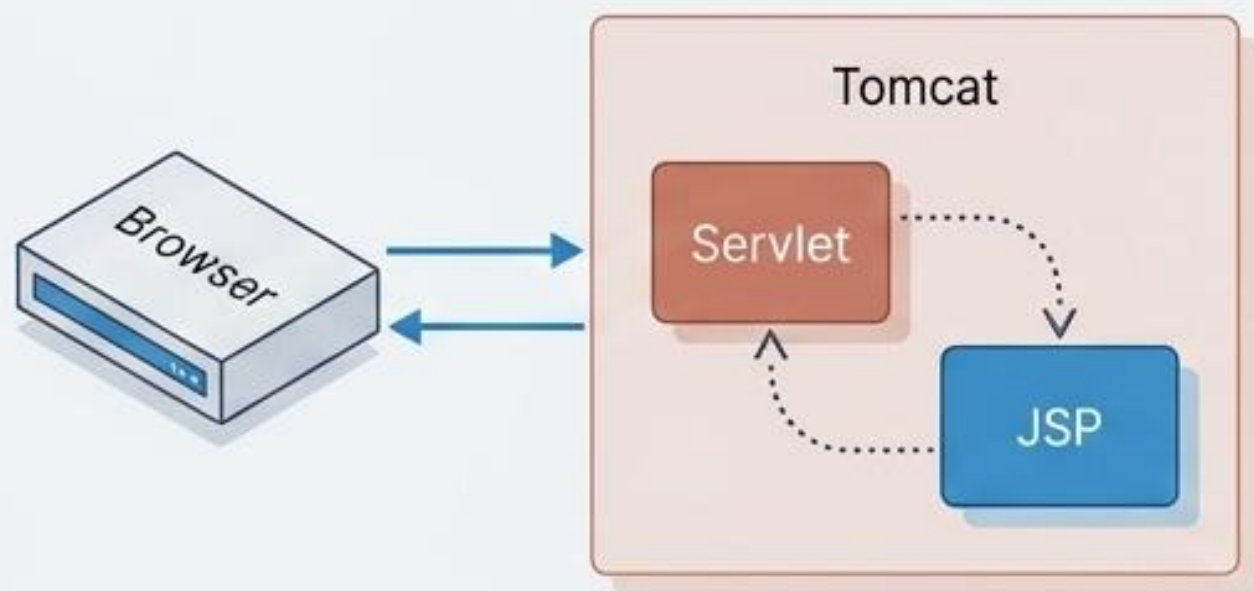


Cookie: 브라우저 티켓

- 저장 위치: 클라이언트 브라우저
- 보안성: 낮음 (변조 가능성 있음)
- 사용 예시: '오늘 하루 보지 않기', 장바구니 임시 저장

```
response.addCookie(new Cookie('id', 'kim'));
```

화면 이동 방식: Forward vs Redirect

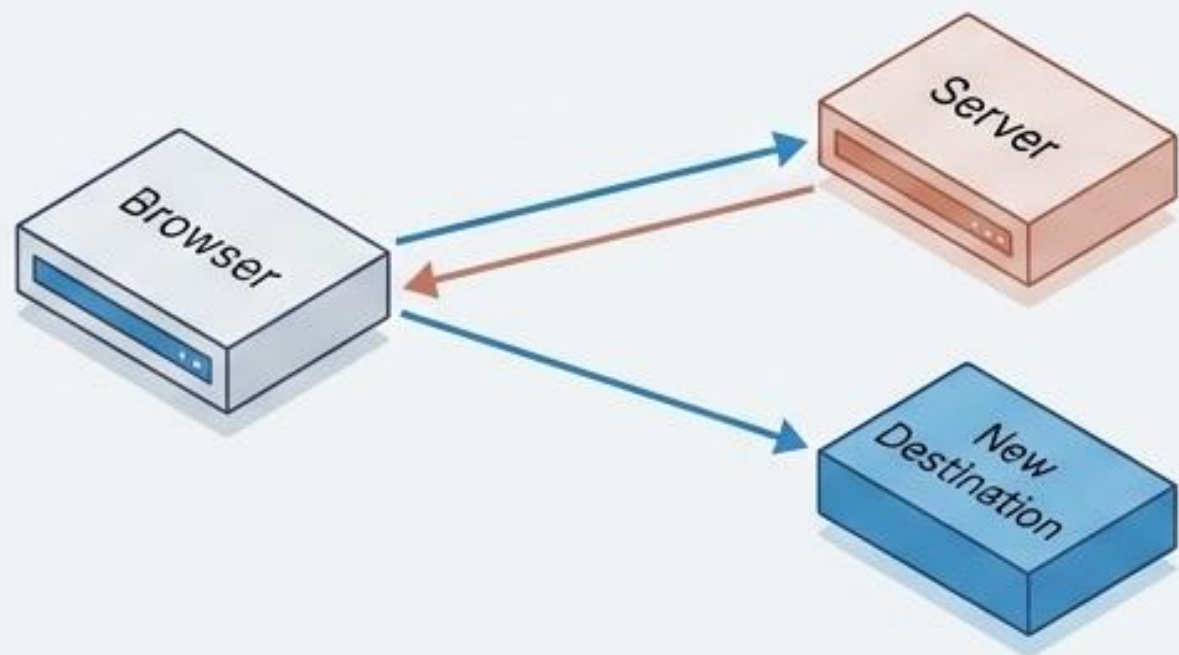


Forward: 서버 내부 전달

특징: URL이 변경되지 않음. 최초의 request/response 객체가 그대로 유지됨.

용도: Servlet에서 데이터 처리 후 JSP로 화면 출력을 넘길 때.

```
rd.forward(request, response);
```



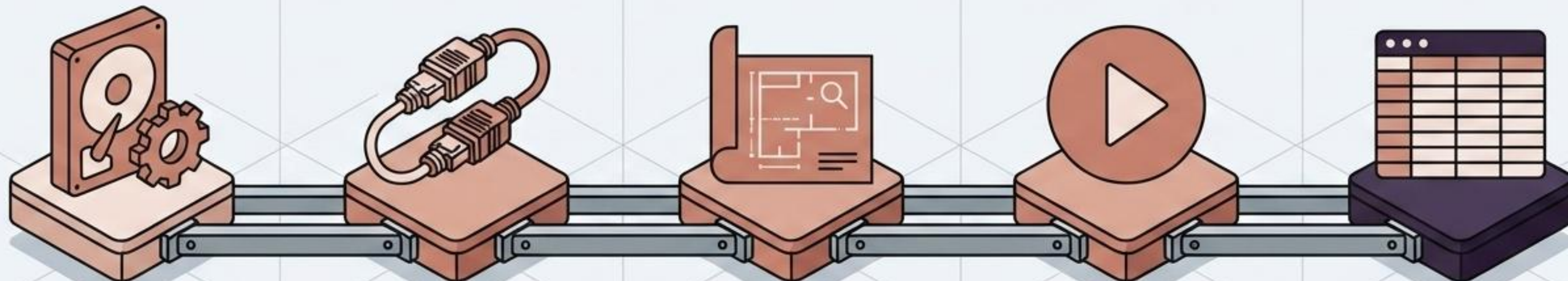
Redirect: 브라우저 재요청

특징: URL이 변경됨. 브라우저가 새로운 요청을 보내므로 기존 request는 초기화됨.

용도: 글쓰기, 결제 완료 후 새로운 페이지로 완전히 이동할 때.

```
response.sendRedirect('main.jsp');
```

데이터베이스 연동: JDBC 5단계 파이프라인



1. Driver 로딩

(DB와 통신하기 위한
드라이버 준비)

```
Class.forName('com.mysql.cj.jdbc.Driver');
```

2. Connection

(DB와 물리적인 연결
통로 생성)

```
DriverManager.getConnection(url, id, pw);
```

3. PreparedStatement

(실행할 SQL 쿼리문 준비)

```
conn.prepareStatement('select * from member');
```

4. executeQuery()

(DB에 쿼리 전송 및 실행)

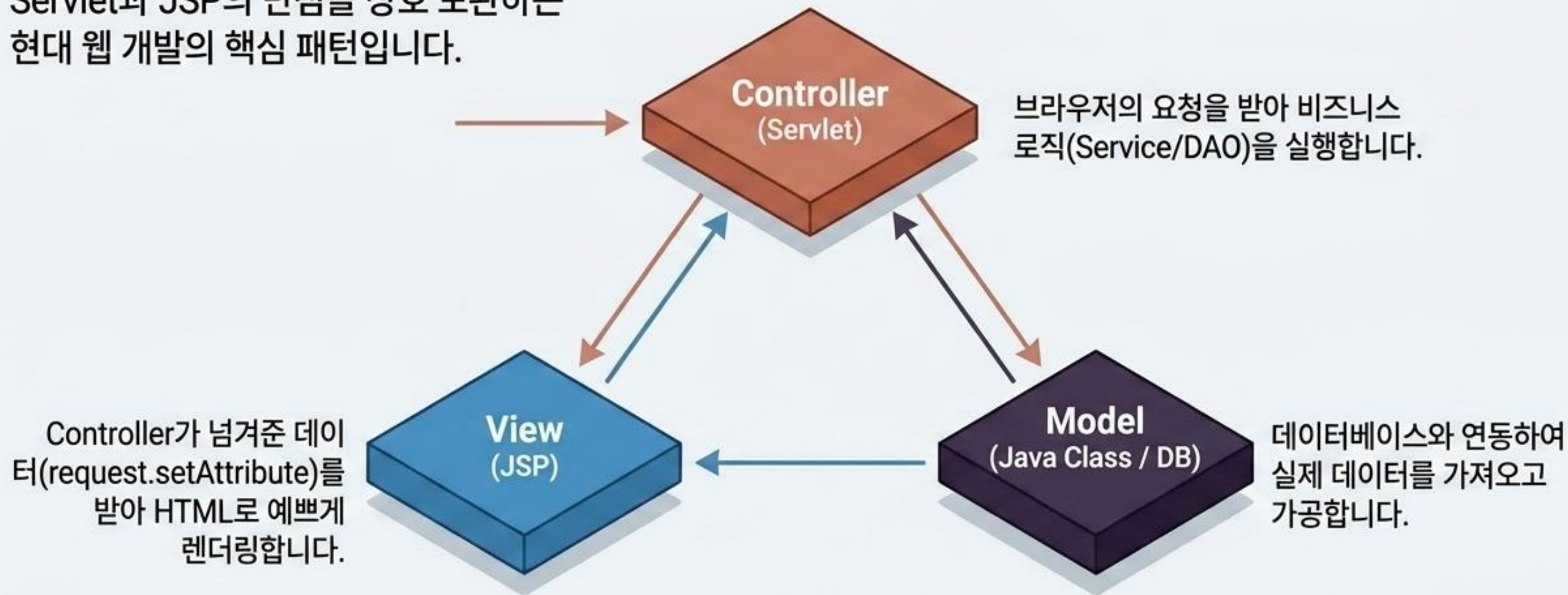
```
ps.executeQuery();
```

5. ResultSet

(DB로부터 반환된 결과
데이터를 자바로 수신)

퍼즐의 완성: MVC 아키텍처 (Model-View-Controller)

Servlet과 JSP의 단점을 상호 보완하는
현대 웹 개발의 핵심 패턴입니다.



로직과 화면의 완벽한 분리

핵심 데이터 흐름 (가장 중요한 마스터 플로우)

1. Client 요청 (브라우저에서 링크 클릭 또는 Form 전송)

```
GET /member HTTP/1.1  
Host: example.com
```

3. Business Logic 처리 (비즈니스 로직 수행, 예: DB 조회)

```
String name = '홍길동';  
// (DB에서 데이터 조회 로직)
```

5. Forward 라우팅 (JSP로 제어권 및 데이터 이관)

```
RequestDispatcher rd = req.getRequestDispatcher('/view.jsp');  
rd.forward(req, resp);
```

7. Response (완성된 HTML이 브라우저로 반환)

2. Servlet (Controller) 개입 (웹 컨테이너가 요청을 가로채 해당 서블릿 실행)

```
@WebServlet('/member')  
public void doGet(req, resp) { ... }
```

4. 데이터 적재 (Scope) (처리 결과를 Request 영역에 저장)

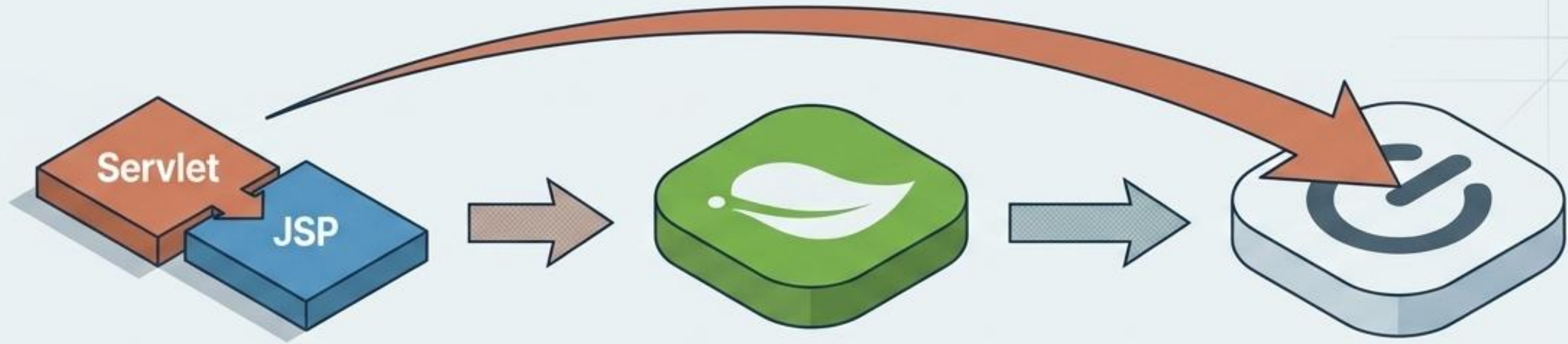
```
req.setAttribute('name', name);  
// (Request 영역에 저장)
```

6. JSP (View) 출력 (EL을 사용해 HTML로 렌더링)

```
<h1>${name}님 환영합니다!</h1>  
<!-- EL을 사용해 데이터 출력 -->
```



실무 아키텍처의 진화: 왜 아직도 Servlet을 배우는가?



[과거]: JSP / Servlet 직접 작성

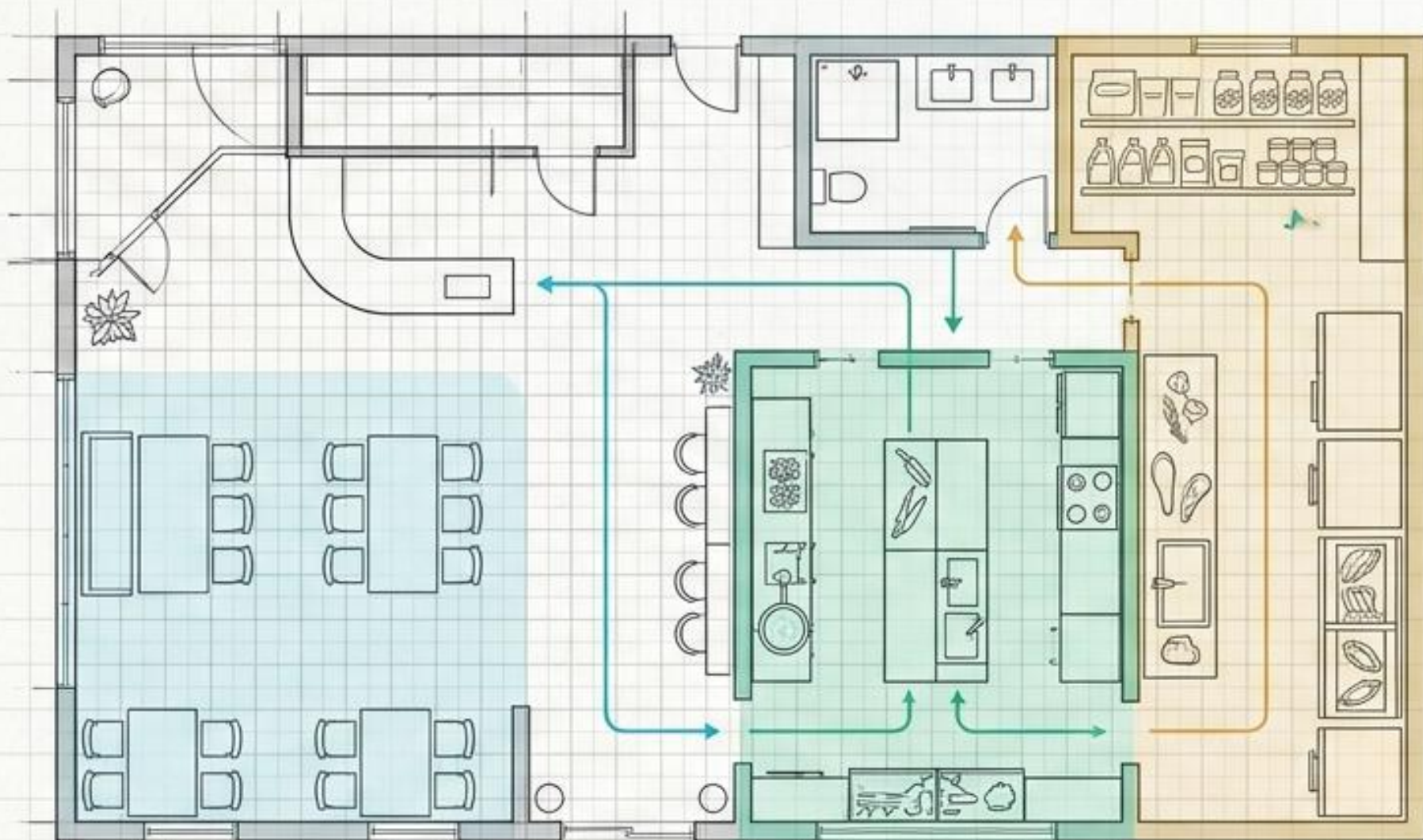
Spring MVC

[현재]: Spring Boot 환경에서 개발

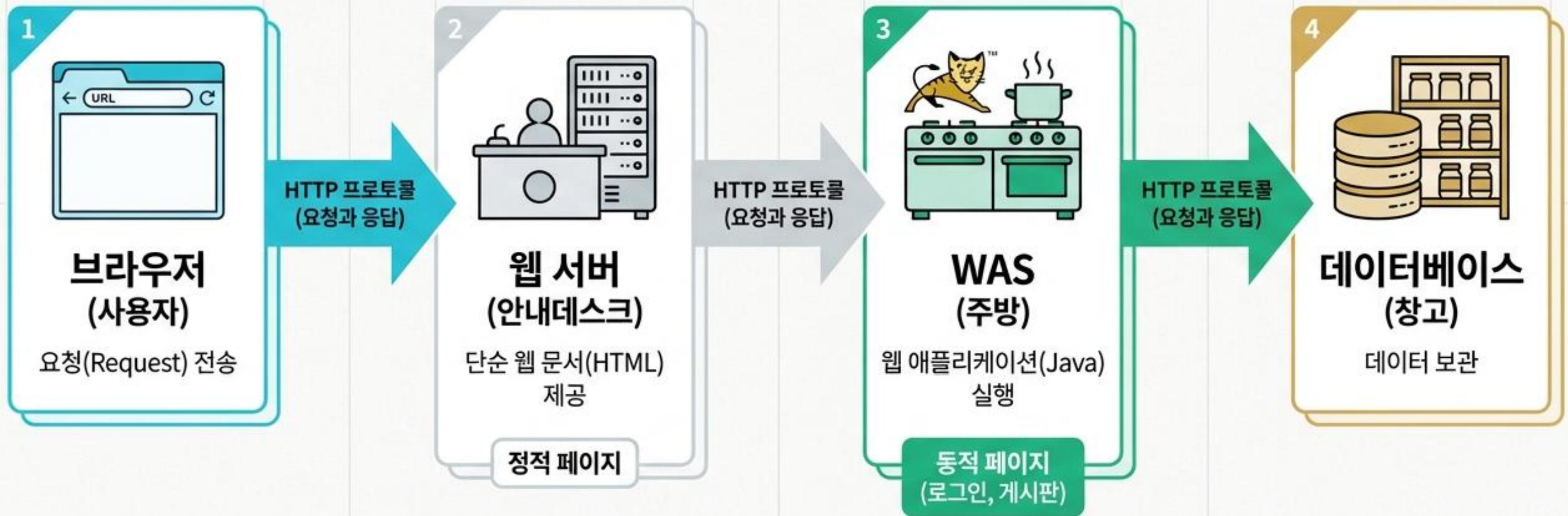
- **진실:** Spring 프레임워크의 심장인 'DispatcherServlet'도 결국 Servlet입니다.
- **현대 프레임워크는 마법이 아닙니다.** Servlet의 생명주기(Lifecycle), Forward/Redirect 라우팅, **Scope**와 **Session** 관리 원리를 내부적으로 **자동화**했을 뿐입니다.
- **기본기**(Servlet/JSP)에 대한 **완벽한 이해**가, 더 높은 수준의 **개발자**(Spring Master)로 가는 **유일한 지름길**입니다.

JSP 웹 프로그래밍: 디지털 레스토랑 구축 가이드

비전공자를 위한 15주차 핵심 요약 플레이북

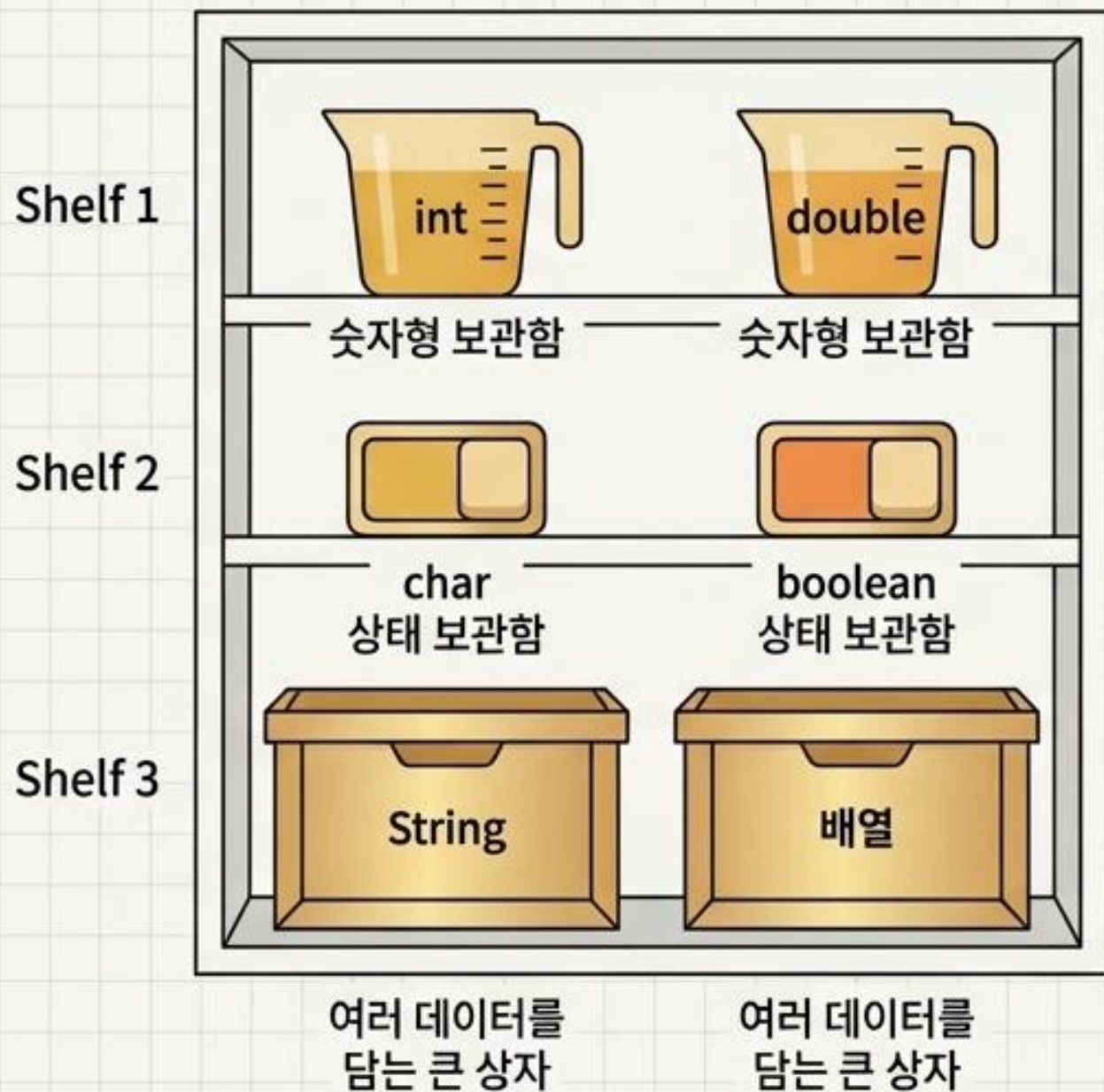


웹 생태계의 물리적 구조

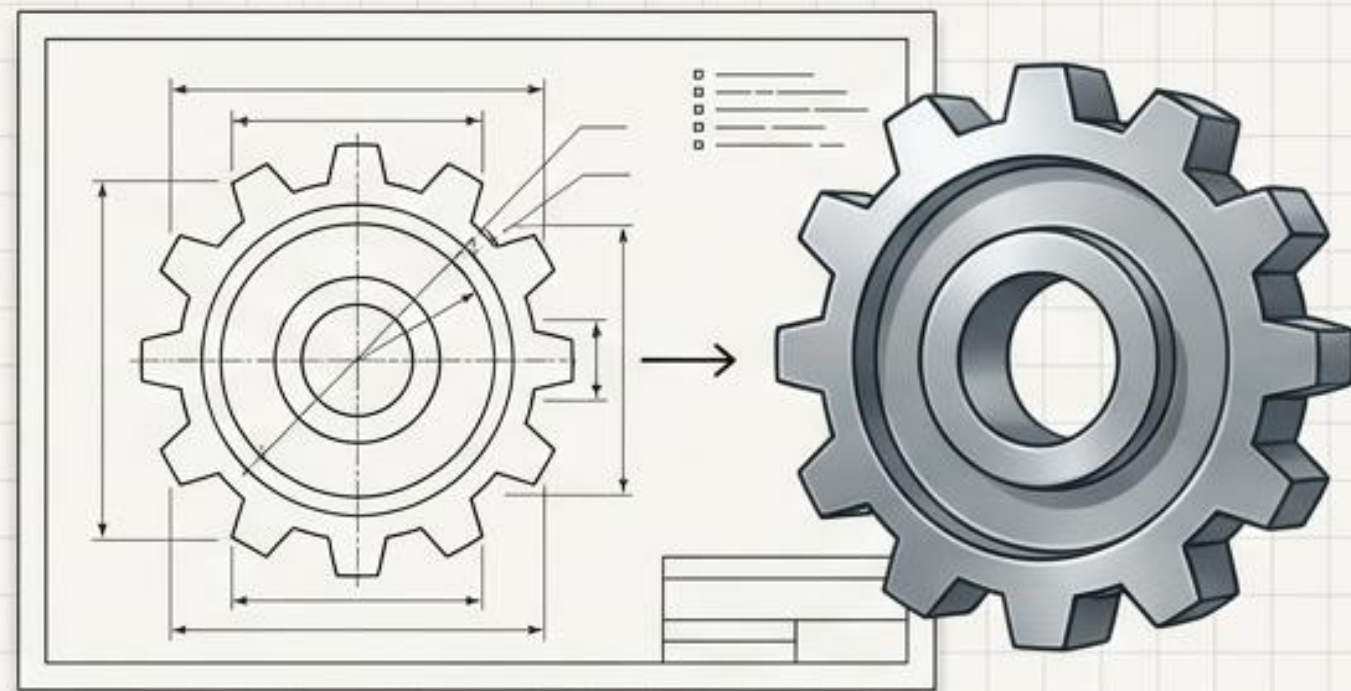


시스템의 기본 재료: Java와 객체지향

The Ingredients



The Rules



클래스 (Class):
설계도 (Blueprint)

객체 (Object):
실제 만들어진 제품 (Product)

상속:
부모 설계도의
기능 재사용

인터페이스:
시스템 간의
필수 규칙 정의

예외 처리 (try/catch):
예상치 못한 오류 발생
시 대처 매뉴얼

직원과 셰프의 역할 분담: JSP vs 서블릿



JSP (홀 직원 / 메뉴판)

역할

화면(View) 생성

형태

HTML + Java (<%@ page %>, <% %>)

난이도

직관적이고 쉬움 (가독성 좋음)

용도

로그인 화면, 결과 출력 페이지



Servlet (주방 셰프)

역할

로직 처리 (Controller)

형태

순수 Java 코드

난이도

비교적 복잡함

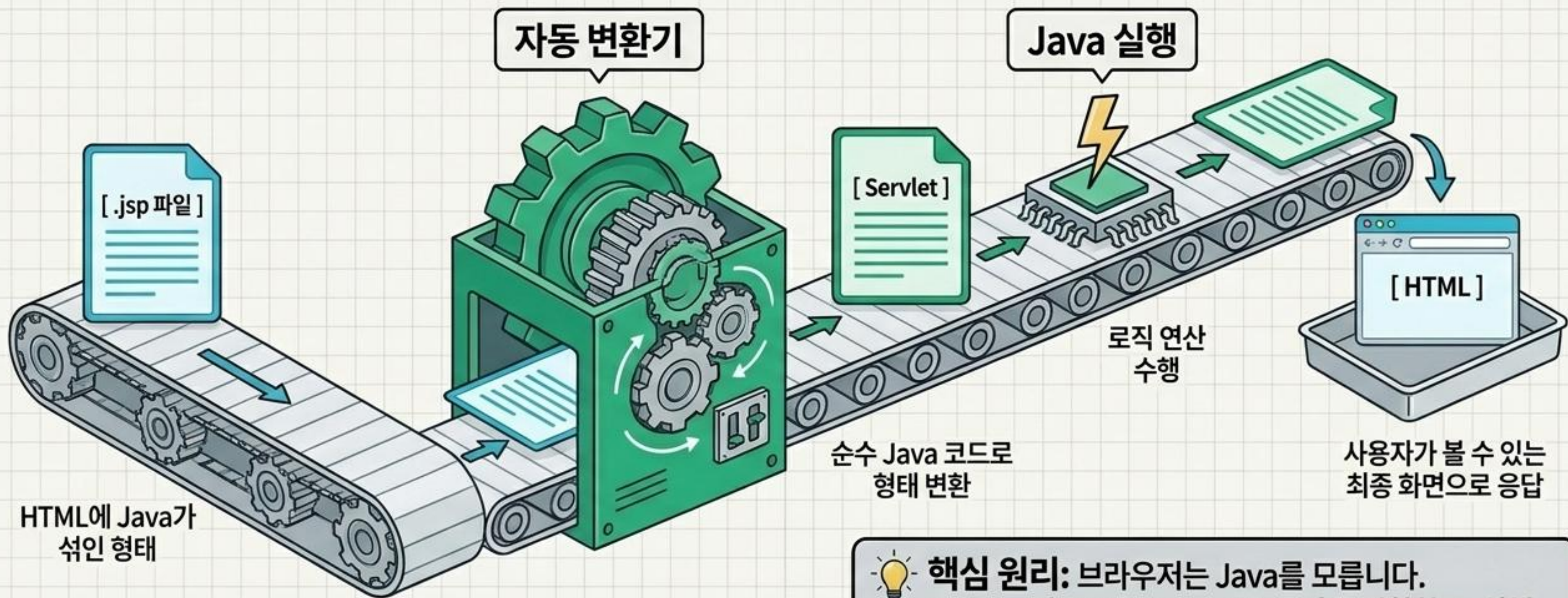
용도

회원가입 처리, DB 연동



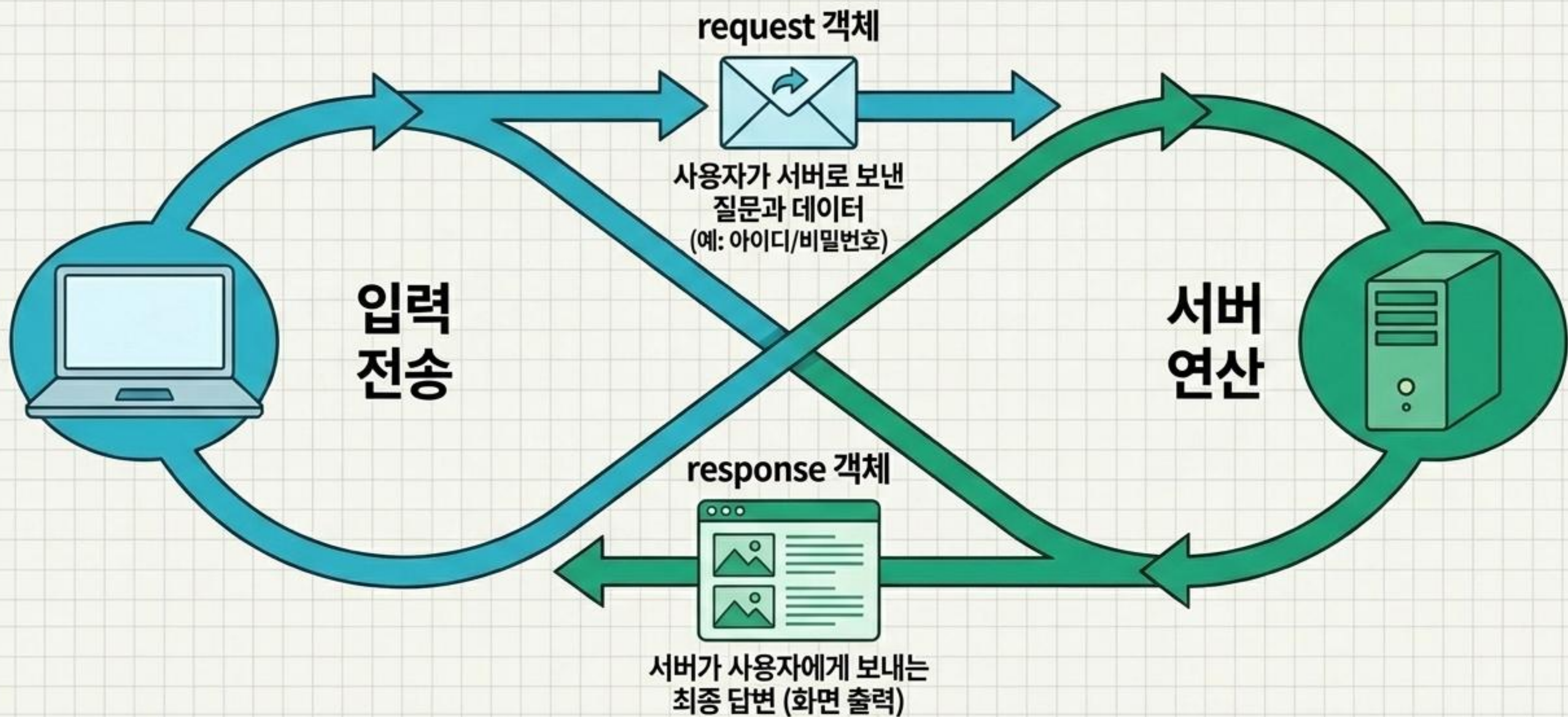
핵심 포인트: JSP는 겉은 HTML, 속은 Servlet. 화면을 쉽게 만들기 위해 포장된 기술.

JSP의 숨겨진 동작 파이프라인



핵심 원리: 브라우저는 Java를 모릅니다.
Tomcat이 JSP를 Servlet으로 자동 변환하고, 실행
결과를 순수 HTML로 만들어 브라우저에 전달합니다.

소통의 순환: 요청(Request)과 응답(Response)



고객 기억법: 쿠키(Cookie) vs 세션(Session)

쿠키 (Cookie)



저장소: 클라이언트
(브라우저)



보안성: 낮음
(위변조 위험)



활용: 최근 본 상품,
'오늘 하루 보지 않기'
팝업

세션 (Session)



저장소: 서버
(Tomcat)



보안성: 매우 높음

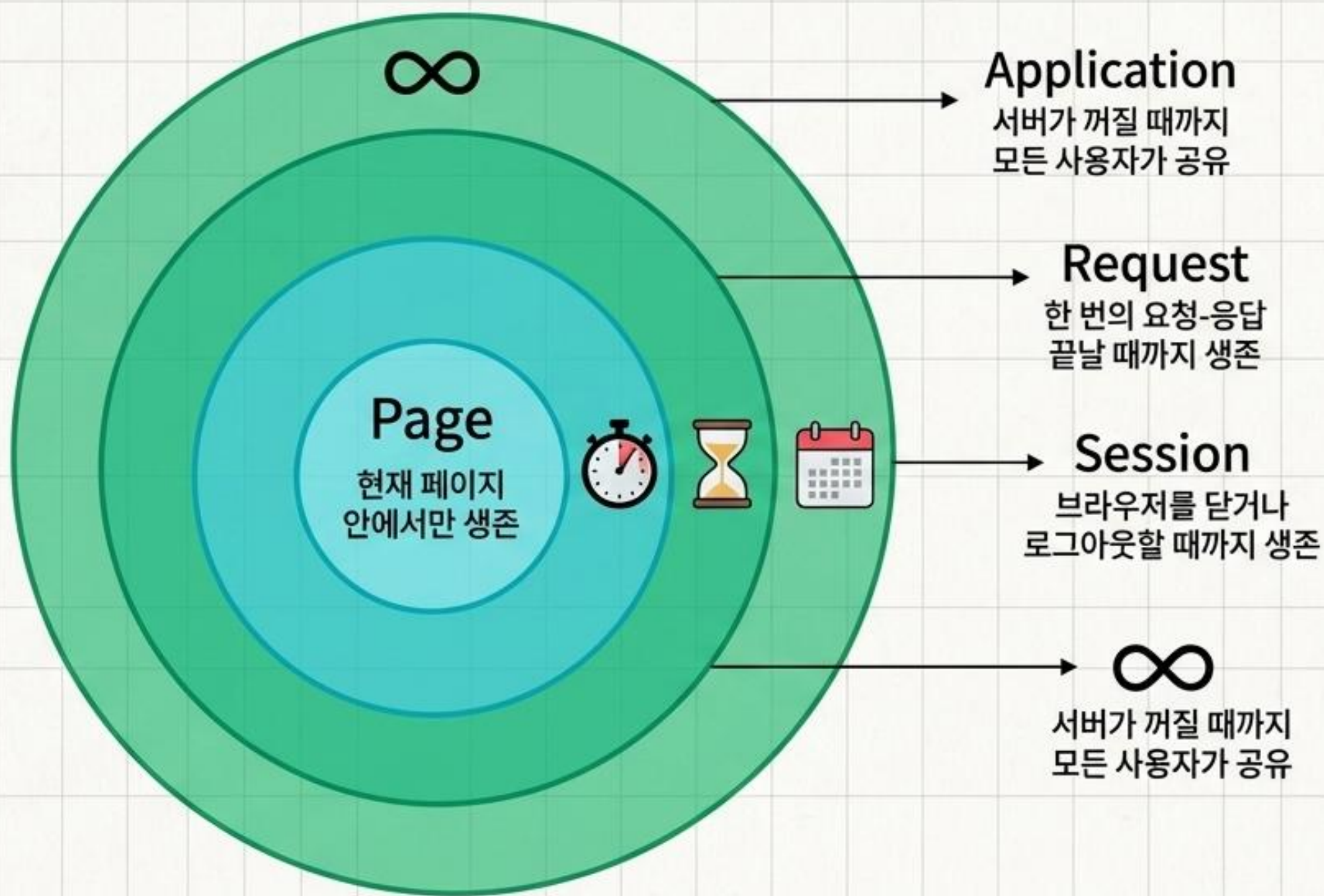


활용:
로그인 상태 유지
(핵심 기술)



주의: 세션은 쿠키보다 안전하지만, 서버의 메모리를
사용하므로 꼭 필요한 곳(로그인)에만 사용합니다.

데이터의 생존 주기 (The 4 Scopes)



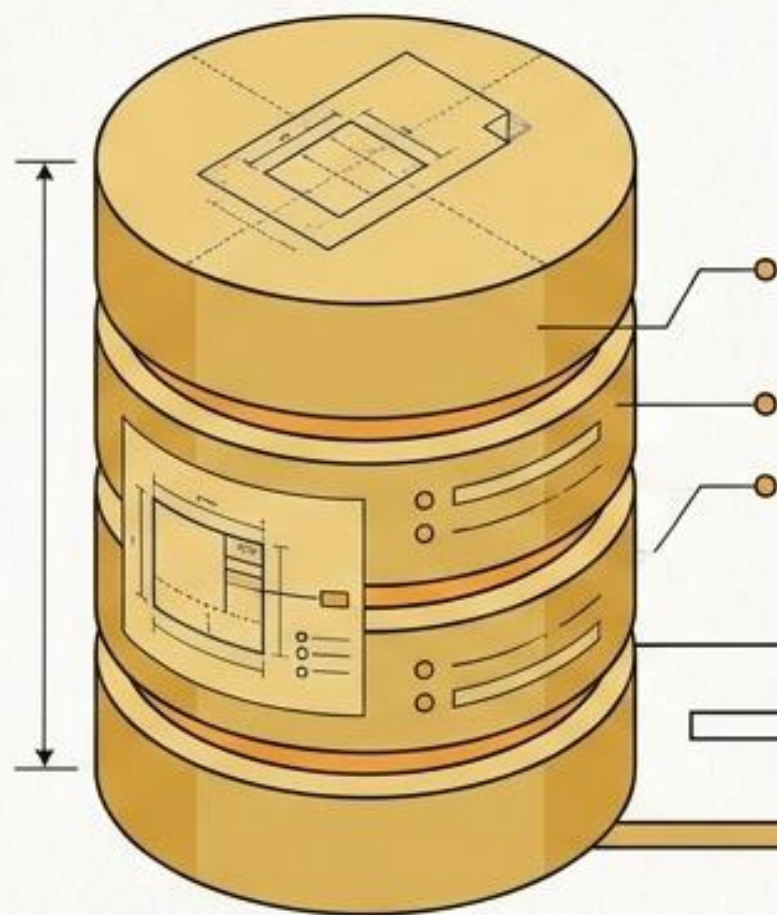
수명이 길고 무거움

수명이 짧고 가벼움

데이터 운반과 식자재 창고 (Java Bean & JDBC)

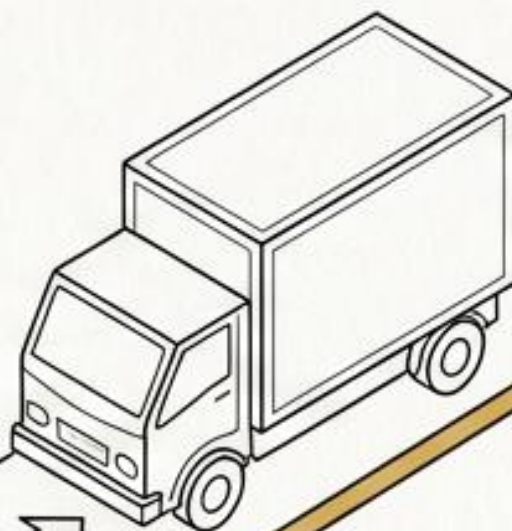
DB (MySQL)

거대한 데이터 창고



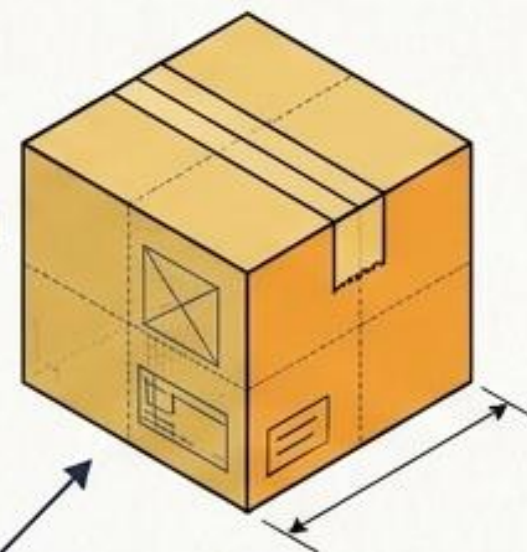
JDBC

창고와 주방을 연결하는 배달 트럭
(DB 연결 -> SQL 실행 -> 결과 받기)



Java Bean

트럭에 실린 규격화된 택배 상자

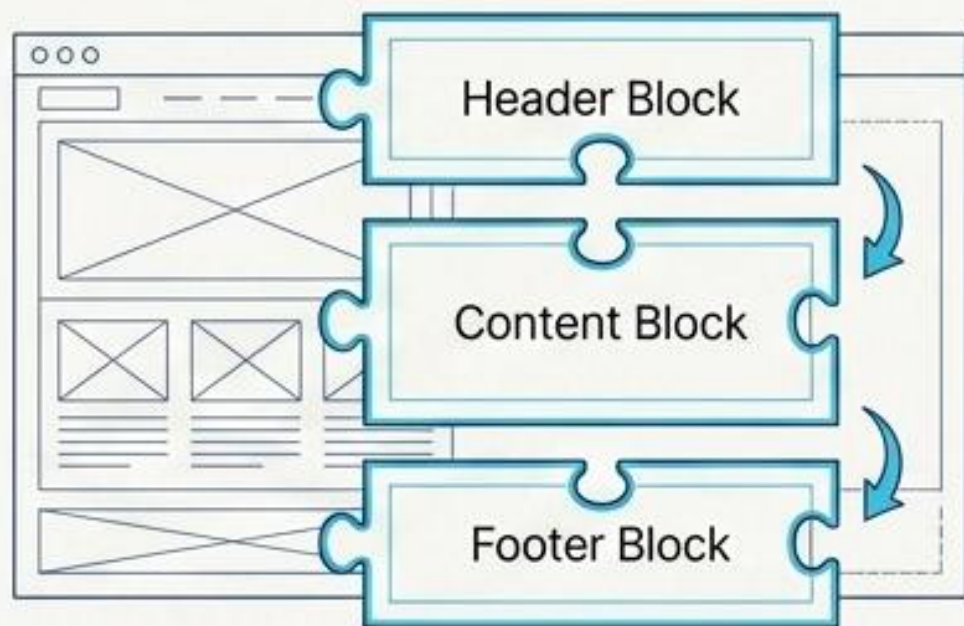


Java Bean 규칙

private 변수 (데이터를 안전하게 숨김)
getter/setter (메서드를 통한 접근 통제)

데이터베이스에서 꺼낸 수많은 회원 정보를 규격화된 객체(Bean)에 담아 JSP/Servlet으로 전달합니다.

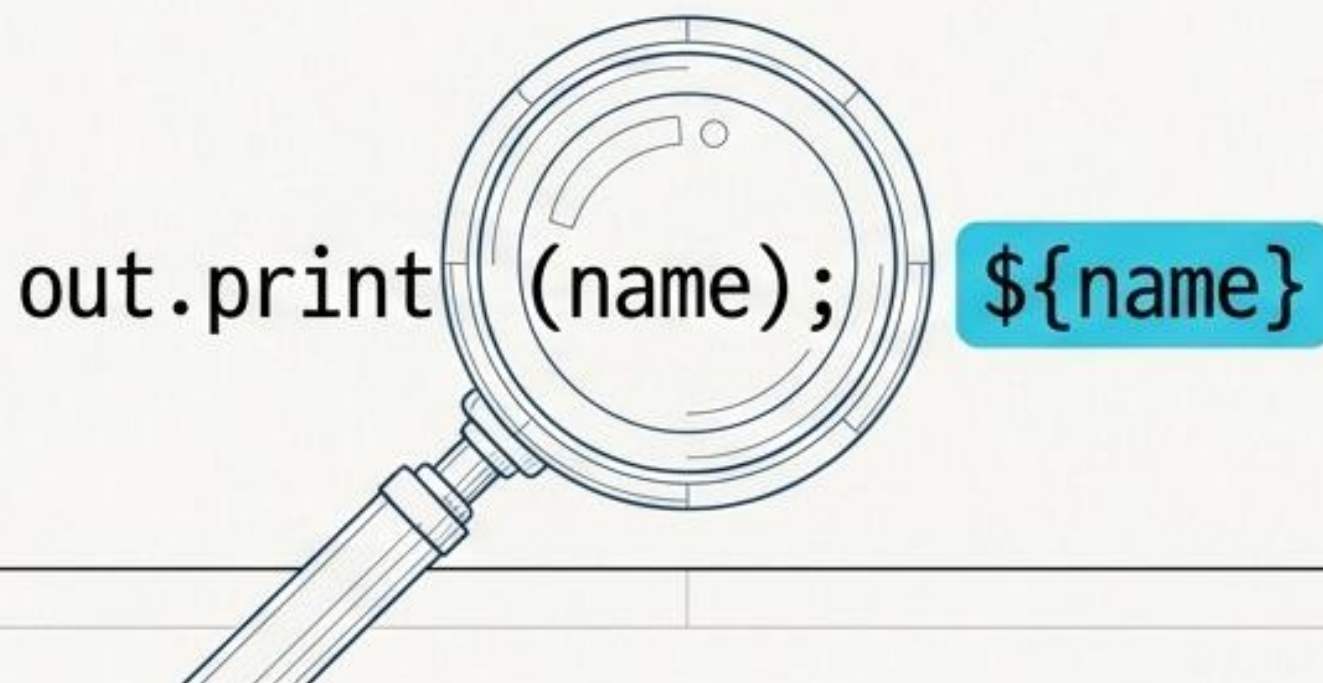
코드 모듈화와 자동화 도구



모듈화와 이동 (Include & Forward)

include: 공통 요소(헤더, 푸터)를 조립식으로 결합

↳ forward: 서버 내부에서 다른 페이지로 자연스럽게 이동

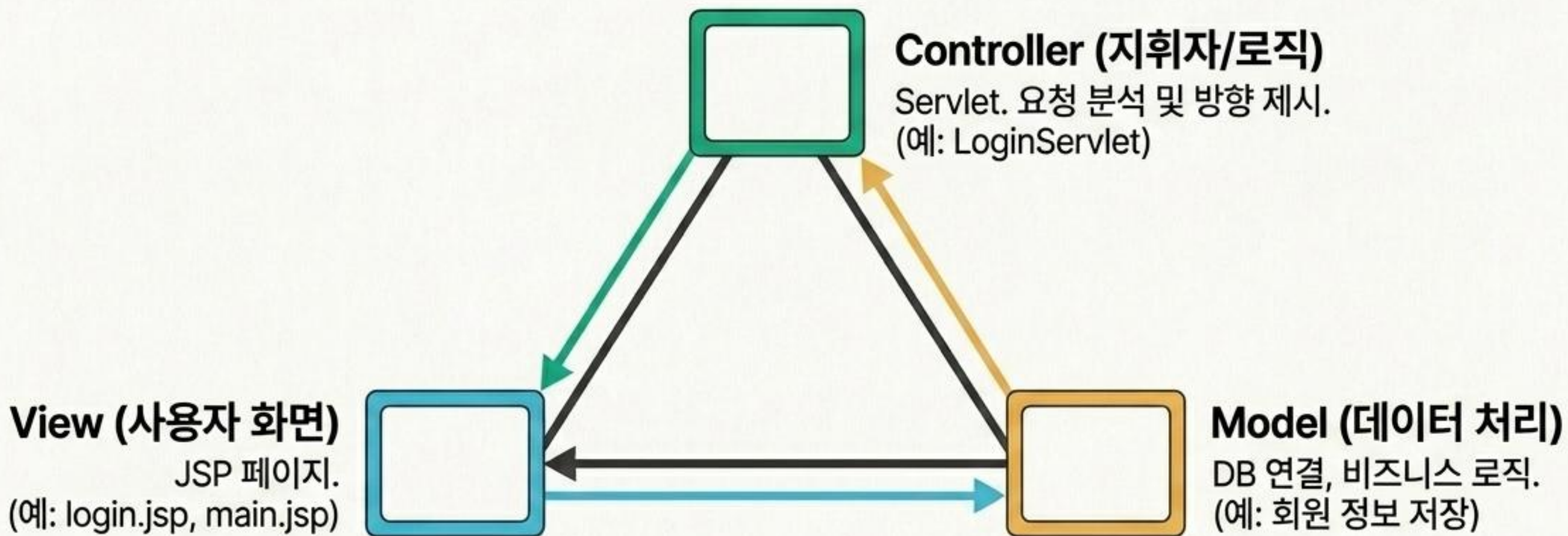


코드 압축 도구 (EL & JSTL)

표현 언어(EL): 복잡한 Java 코드 없이 `${}` 만으로 데이터 출력

JSTL: JSP 내부에 반복문, 조건문 등을 HTML 태그처럼 쉽게 작성

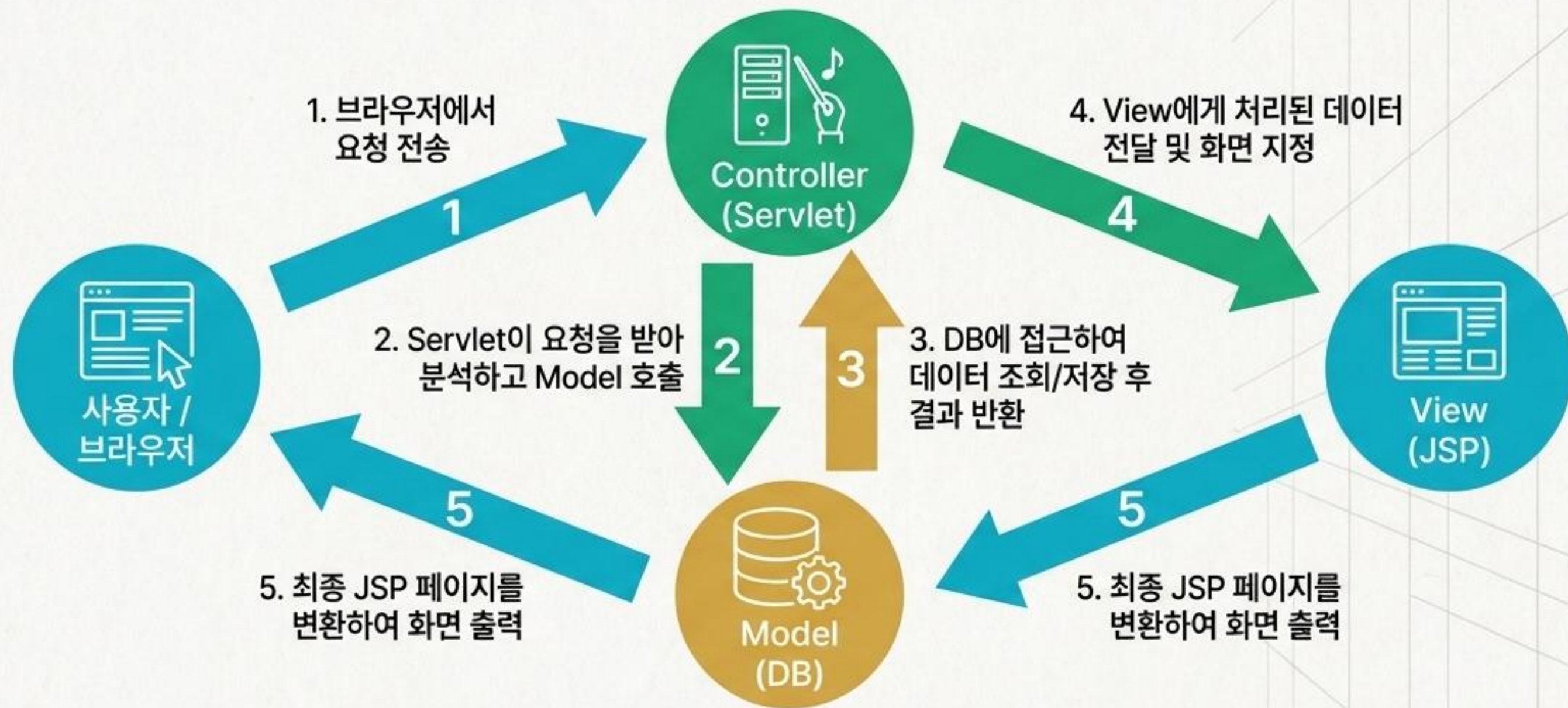
마스터 아키텍처: MVC 패턴 도입



Key Insight

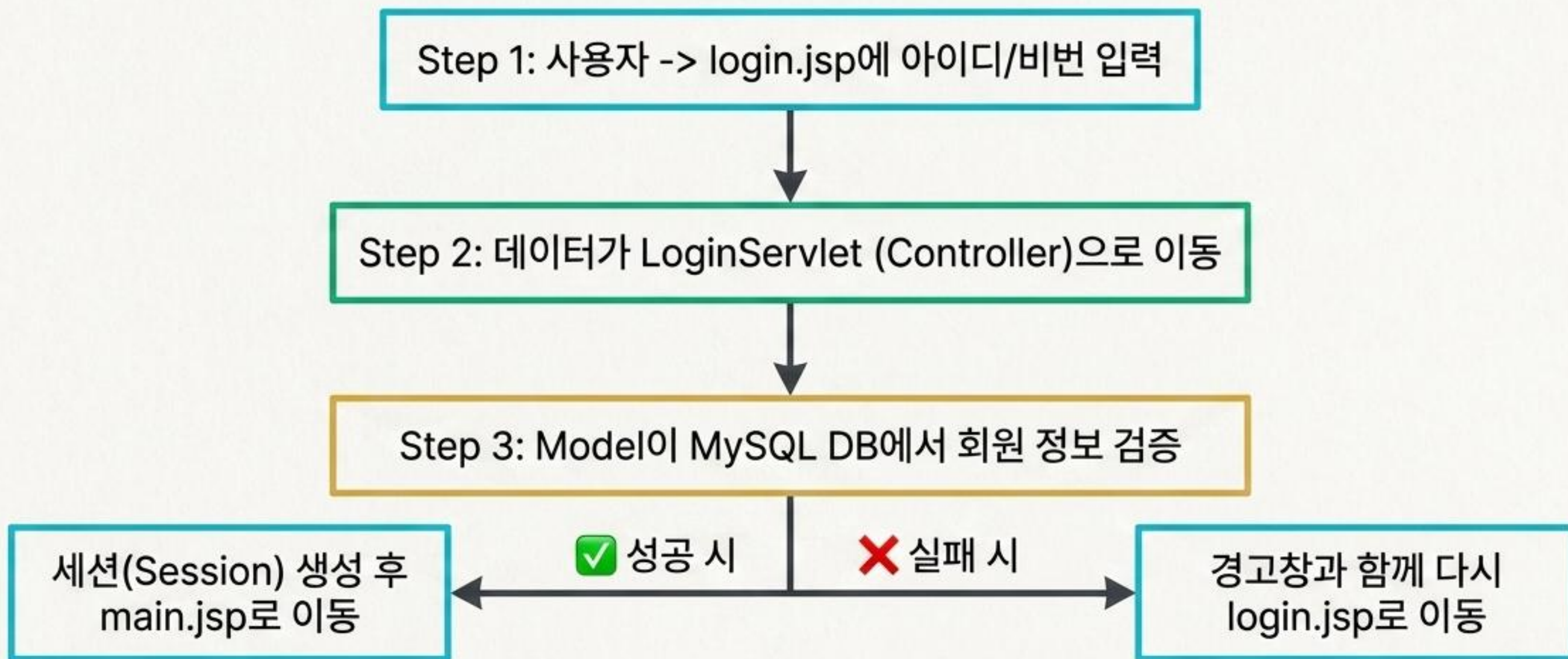
View는 화면 출력만, Model은 데이터 처리만, Controller는 길 안내만.
철저한 역할 분리로 유지보수가 쉬워집니다.

MVC 시스템 동작 플로우



이 거대한 순환 구조가 바로 현대 웹 애플리케이션이 끊임없이 동작하는 원리입니다.
단독으로 쓰이던 JSP와 Servlet은 MVC 안에서 완전한 균형을 찾습니다.

실전 플로우 추적: 로그인 시스템



로그인 한 번의 클릭 뒤에 View, Controller, Model의 완벽한 협업이 일어납니다.

파이널 프로젝트: 회원 관리 시스템 완공



15주간 배운 모든 기술(JSP, Servlet, DB, MVC)이 결합되어 스스로 작동하는 완벽한 하나의 웹 서비스를 만들어냅니다.

JSP 웹 프로그래밍 치트시트

[Q1] 핵심 기술

- **JSP**: 화면 담당 (HTML + Java)
- **Servlet**: 서버 로직 담당 (순수 Java)
- **JDBC**: 데이터베이스 연결

[Q2] 상태 유지

- **쿠키 (Cookie)**: 브라우저 저장, 보안 낮음
- **세션 (Session)**: 서버 저장, 보안 높음 (로그인 유지)

[Q3] 데이터 생명주기



안쪽일수록 짧고 가벼우며, 바깥일수록 길고 무거움

[Q4] MVC 아키텍처

- **Model**: 데이터베이스와 통신, 로직 연산
- **View**: 최종적으로 보여지는 화면 (JSP)
- **Controller**: 흐름 제어, 길 안내 (Servlet)

이 한 장의 설계도만 기억한다면, 어떤 웹 시스템도 분석하고 만들어낼 수 있습니다.